**Mawlana Bhashani Science and Technology University**
**Santosh,Tangail**
Department of Information & Communication Technology

# SmartShelf

Library Management System with Barcode Circulation, Reservation Queue, Automated Fines and SMS Reminders

## Part 1: Schedule, UML Models and Prototype Interface

**Submitted by:** Md Obayet Khan
**Student ID** : IT24003
**Course** : ICT3108 – Software Engineering & Project Management
**Date** : October 2026

## Project Overview

**SmartShelf** is a desktop School Library Management System. It automates cataloguing, barcode-based issue and return, fine calculation (BDT 5 per late day), a first-come-first-served reservation queue, e-book access tracking and SMS-style overdue reminders. This first part of the project contains: (1) the schedule, milestones, deliverables and Gantt chart; (2) the six UML models; and (3) a working prototype interface built from the designed classes. The remaining proposal sections (feasibility, requirements, risks, test plan) will be added in later parts.

# Contents

**Project Overview** ..... 1

**1 Software Process Model** ..... 4

1.1 Choice: Agile (Scrum) ..... 4

1.2 Sprint Backlog Summary ..... 4

**2 Project Schedule, Milestones and Deliverables** ..... 4

2.1 Work Breakdown and Effort ..... 4

2.2 Gantt Chart ..... 5

2.3 Milestones ..... 6

2.4 Deliverables ..... 6

**3 System Analysis and Design (UML Models)** ..... 7

3.1 Data Flow Diagram ..... 7

3.2 Use Case Diagram ..... 8

3.3 Activity Diagram ..... 9

3.4 Entity Relationship Diagram ..... 10

3.5 Class Diagram ..... 11

3.6 Object Diagram ..... 12

3.7 Architecture ..... 13

**4 Prototype Implementation** ..... 13

4.1 Technology and Structure ..... 13

4.2 Classes Used by the Interface ..... 13

4.3 Screenshots ..... 14

4.4 Selected Source Code ..... 15

# List of Figures

1 Gantt chart for the 14-week Scrum schedule (milestones at weeks 1, 2, 5, 8, 11, 13 and 14). ..... 4

2 Data Flow Diagram, Level 0 (context diagram). ..... 6

3 Data Flow Diagram, Level 1. ..... 7

4 Use Case Diagram. ..... 8

5 Activity Diagram: issue and return of a book. ..... 9

6 Entity Relationship Diagram. ..... 10

7 Class Diagram. . . . . 11
8 Object Diagram: snapshot of the library on 25 October 2026. . . . . 12
9 Four-layer architecture of SmartShelf. . . . . 12
10 Dashboard: KPI cards, books by category, overdue books. . . . . 13
11 Catalog tab: search and add books. . . . . 14
12 Circulation tab: issue book and return selected loan. . . . . 14

# 1 Software Process Model

## 1.1 Choice: Agile (Scrum)

SmartShelf is built with **Scrum**, an iterative and incremental model. Requirements are grouped into a product backlog, delivered in four sprints, and each sprint ends with a working increment that the librarian (as product owner) can review. **Why Scrum rather than Waterfall?**

The librarian's needs become clearer when she sees a working screen; frequent demos reduce rework.

Risk is lowered because the riskiest rules (circulation and fines) are built and tested early.

Each increment is usable, so something can be submitted even if time runs short.

A short *Sprint 0* is added at the start for planning, requirements and design, so the project still has the up-front documentation expected in a large project.

| Scrum element | How it is applied | Frequency |
|---|---|---|
| Product backlog | User stories prioritised by MoSCoW | Updated weekly |
| Sprint planning | Select stories and estimate hours | Start of each sprint |
| Daily check-in | Self review: done, next, blockers (kept in a log) | Daily (5 min) |
| Sprint review/demo | Show the increment to the librarian/instructor | End of each sprint |
| Retrospective | What to keep, stop and improve | End of each sprint |
| Definition of Done | Code works, unit tests pass, docs updated, demo accepted | Every story |

**Table 1:** Scrum practices for this project.

## 1.2 Sprint Backlog Summary

| Sprint | Weeks | Goal | Increment delivered |
|---|---|---|---|
| Sprint 0 | 1–2 | Plan, specify and design | Proposal, requirements, six UML diagrams, database design |
| Sprint 1 | 3–5 | Catalogue and members | Add/search books, register members, SQLite schema |
| Sprint 2 | 6–8 | Circulation and fines | Issue, return, loan limits, fine engine |
| Sprint 3 | 9–11 | Reservations, alerts, UI | Reservation queue, SMS log, e-books, dashboard, login |
| Sprint 4 | 12–14 | Test, harden, deliver | System and user acceptance tests, reports, final documents |

**Table 2:** Sprint plan.

# 2 Project Schedule, Milestones and Deliverables

## 2.1 Work Breakdown and Effort

| WBS | Work package | Hours |
|---|---|---|
| 1 | Project management (planning, sprint meetings, tracking) | 14 |
| 2 | Requirements and proposal | 16 |
| 3 | Analysis and design (6 UML diagrams, database design) | 24 |

| | | |
|---|---|---|
| 4 | Implementation: catalogue and members | 20 |
| 5 | Implementation: circulation and fines | 24 |
| 6 | Implementation: reservations, notifications, e-books | 20 |
| 7 | User interface and dashboard | 18 |
| 8 | Testing (unit, system, acceptance) | 18 |
| 9 | Documentation and final presentation | 16 |
| | **Total** | **170** |

**Table 3:** Effort estimate (about 12 hours per week for 14 weeks).

## 2.2 Gantt Chart

Week 1 starts on 1 November 2026; week 14 ends on 6 February 2027. Diamonds mark milestones.

**Sprint 0: Plan & Design**

Proposal, requirements

UML diagrams, DB design

**Sprint 1: Catalogue & Members**

Catalogue module (books, search)

Member module

Sprint 1 test and demo

**Sprint 2: Circulation & Fines**

Issue / return logic

Fine engine and loan limits

Sprint 2 test and demo

**Sprint 3: Reservations, Alerts, UI**

Reservation queue

SMS alerts and e-books

Dashboard UI and login Sprint 3 test and demo

**Sprint 4: Test & Deliver**

System and acceptance testing

Bug fixing, reports, polish

Final report and presentation

Milestones M1–M7

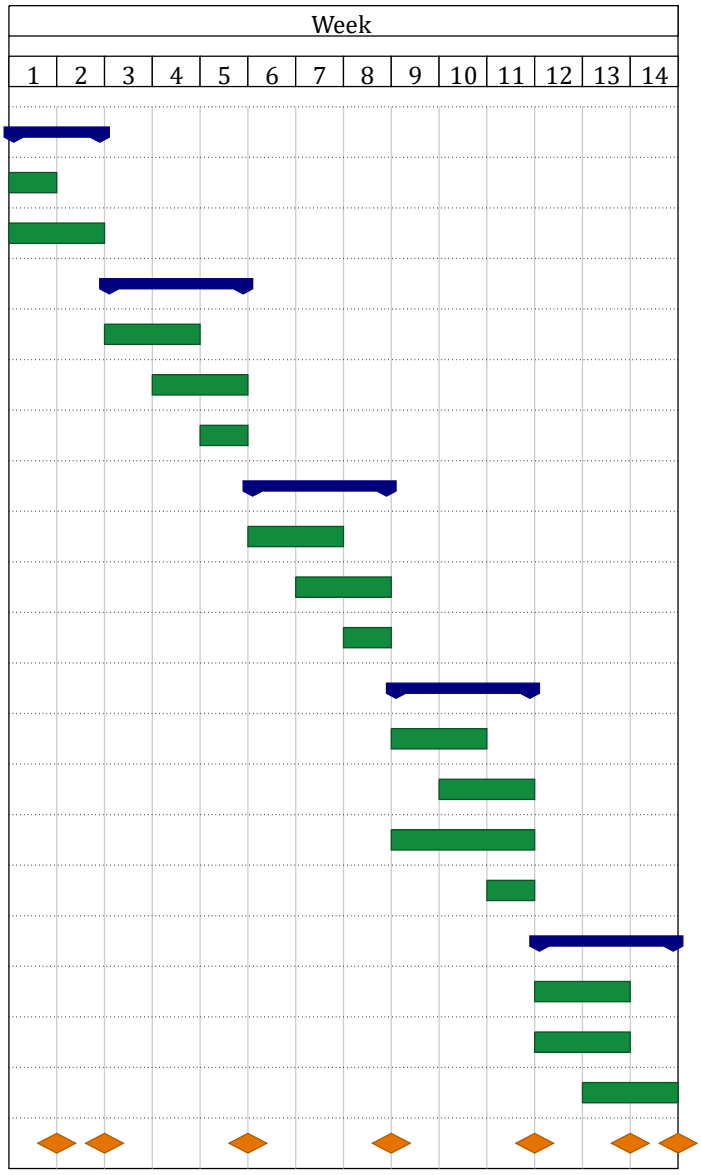



**Figure 1:** Gantt chart for the 14-week Scrum schedule (milestones at weeks 1, 2, 5, 8, 11, 13 and 14).

## 2.3 Milestones

| ID | Week | Milestone and acceptance criterion | Date |
|---|---|---|---|
| M1 | 1 | Proposal and requirements approved by the instructor | 7 Nov 2026 |
| M2 | 2 | Design baseline: six UML diagrams and schema reviewed | 14 Nov 2026 |
| M3 | 5 | Sprint 1 demo: books and members can be added and searched | 5 Dec 2026 |
| M4 | 8 | Sprint 2 demo: issue, return and fines pass all rule tests | 26 Dec 2026 |
| M5 | 11 | Sprint 3 demo: reservations, alerts, e-books and dashboard | 16 Jan 2027 |
| M6 | 13 | Release candidate: all tests pass, user acceptance sign-off | 30 Jan 2027 |
| M7 | 14 | Final submission and presentation | 6 Feb 2027 |

**Table 4:** Project milestones.

## 2.4 Deliverables

| ID | Deliverable | Description | Due |
|---|---|---|---|

| | | | |
|---|---|---|---|
| D1 | Project proposal | Full proposal text (to be completed in a later part) | Week 1 |
| D2 | Requirements specification | Functional and non-functional requirements | Week 1 |
| D3 | Process plan | Process model, WBS, schedule, Gantt chart | Week 1 |
| D4 | UML design set | DFD, use case, activity, ER, class, object diagrams | Week 2 |
| D5 | Database schema | SQLite DDL with 9 tables | Week 2 |
| D6 | Increment 1 | Catalogue and member modules + tests | Week 5 |
| D7 | Increment 2 | Circulation and fine modules + tests | Week 8 |
| D8 | Increment 3 | Reservations, alerts, e-books, dashboard | Week 11 |
| D9 | Test report | Test plan, cases, results, defect log | Week 13 |
| D10 | Release 1.0 | Source code, demo database, user guide | Week 13 |
| D11 | Final report and slides | Complete LaTeX report and presentation | Week 14 |

**Table 5:** Project deliverables.

# 3 System Analysis and Design (UML Models)

The models are consistent with each other: the actors of the use case diagram appear as external entities in the data flow diagram, the data stores D1–D7 correspond to ER entities, and the ER entities correspond to the classes in the class diagram.

## 3.1 Data Flow Diagram

The context diagram (Level 0) shows the system boundary and the four external entities. Level 1 breaks the system into eight processes and seven data stores.

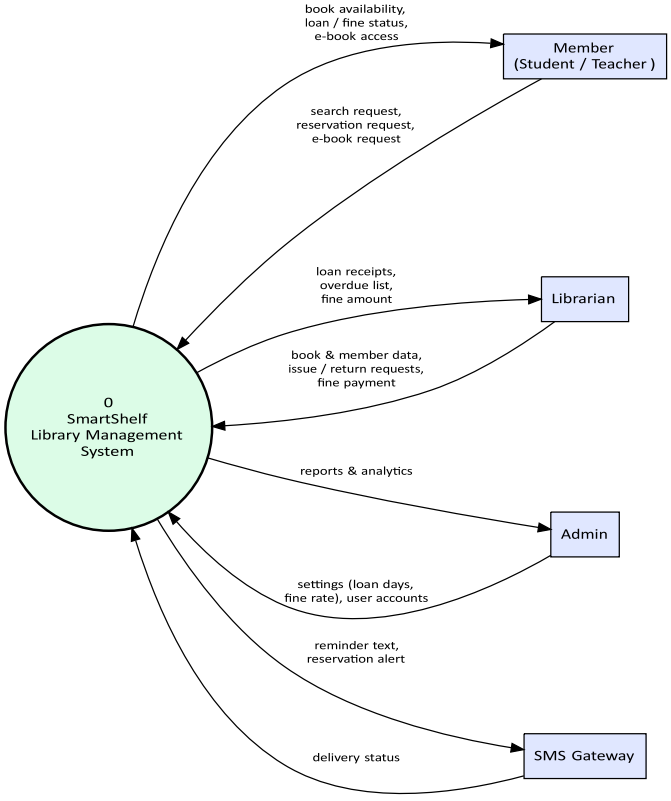



**Figure 2:** Data Flow Diagram, Level 0 (context diagram).

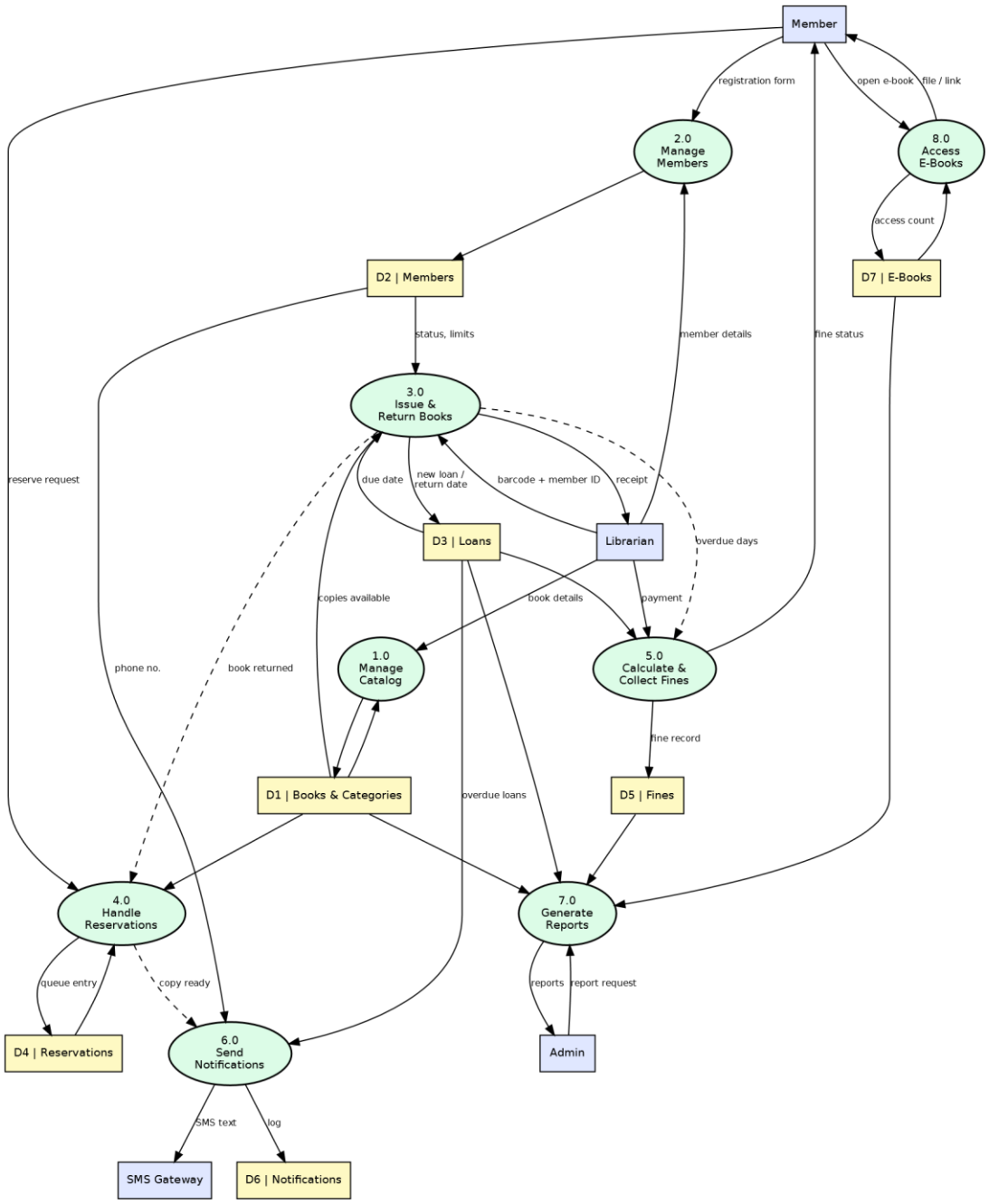



**Figure 3:** Data Flow Diagram, Level 1.

## 3.2 Use Case Diagram

Three human actors (Member, Librarian, Admin) and one system actor (SMS Gateway) interact with 15 use cases. Yellow use cases are included or extending behaviour.

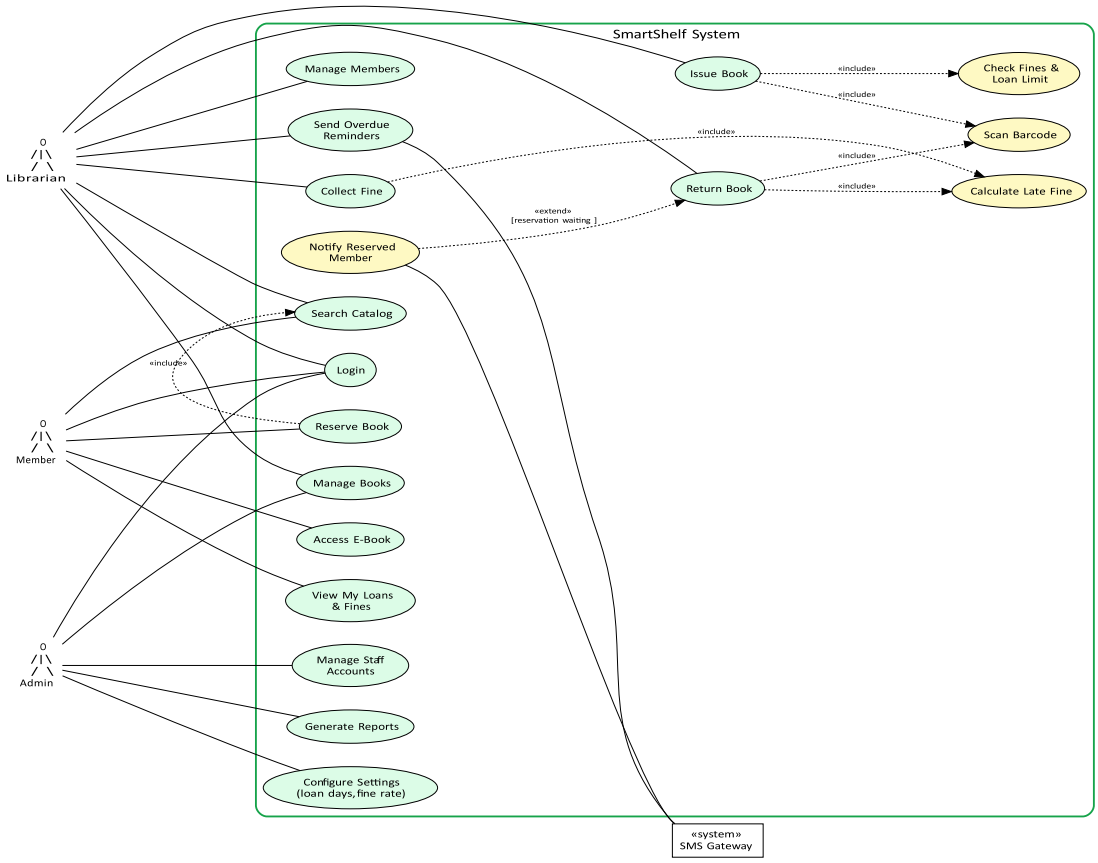



**Figure 4:** Use Case Diagram.

| Use case | Actor | Brief description / main flow |
|---|---|---|
| UC-01 Issue Book | Librarian | Scan member ID and book barcode; the system checks status, fines, loan limit and copy availability, then creates a loan with a due date. |
| UC-02 Return Book | Librarian | Select active loan; the system closes it, adds a fine when late, restores the copy and promotes the next reservation. |
| UC-03 Reserve Book | Member / Librarian | Choose a title with no free copy; the member joins the queue. |
| UC-04 Collect Fine | Librarian | Select an unpaid fine and mark it paid. |
| UC-05 Send Reminders | Librarian | Create SMS messages for all overdue loans. |
| UC-06 Generate Reports | Admin | View circulation, overdue and fine statistics. |

**Table 6:** Key use case descriptions.

## 3.3 Activity Diagram

The diagram models the complete life of a loan: validation and issue, the loan period, and return with fine calculation and reservation handling. Diamonds are decisions; red actions are rejections.

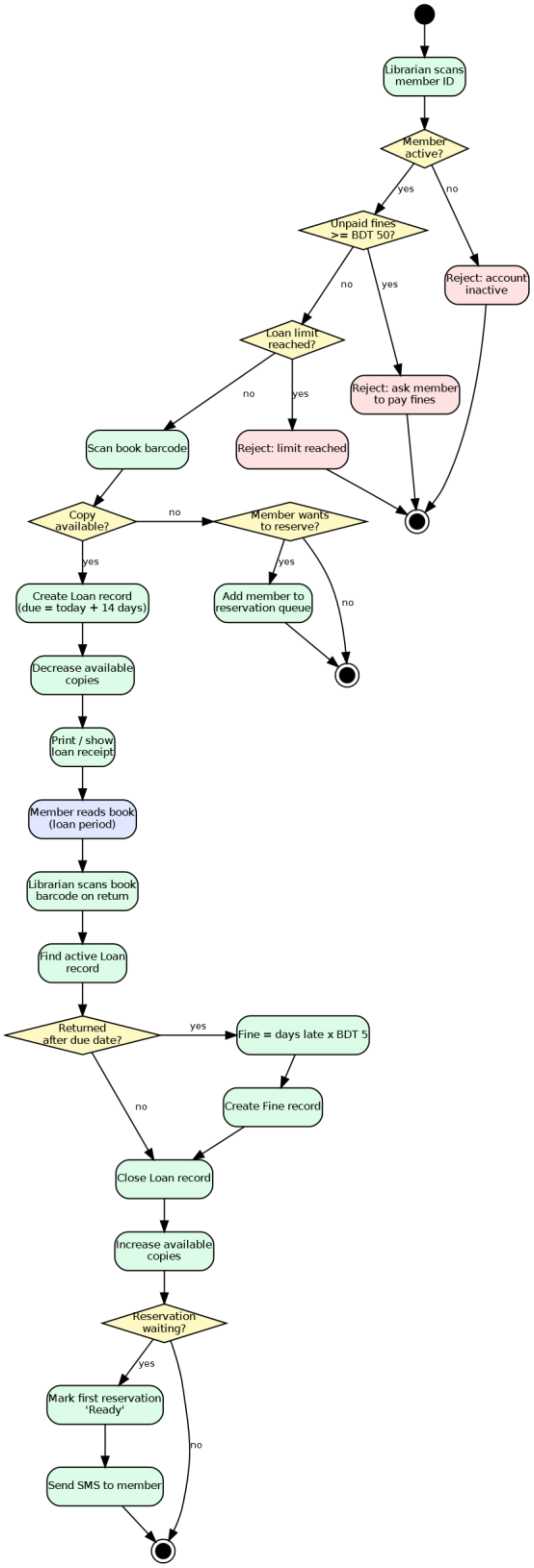



**Figure 5:** Activity Diagram: issue and return of a book.

## 3.4 Entity Relationship Diagram

The database has nine tables. Crow's-foot symbols show cardinality: for example, one member can have many loans, and one loan may produce at most one fine.

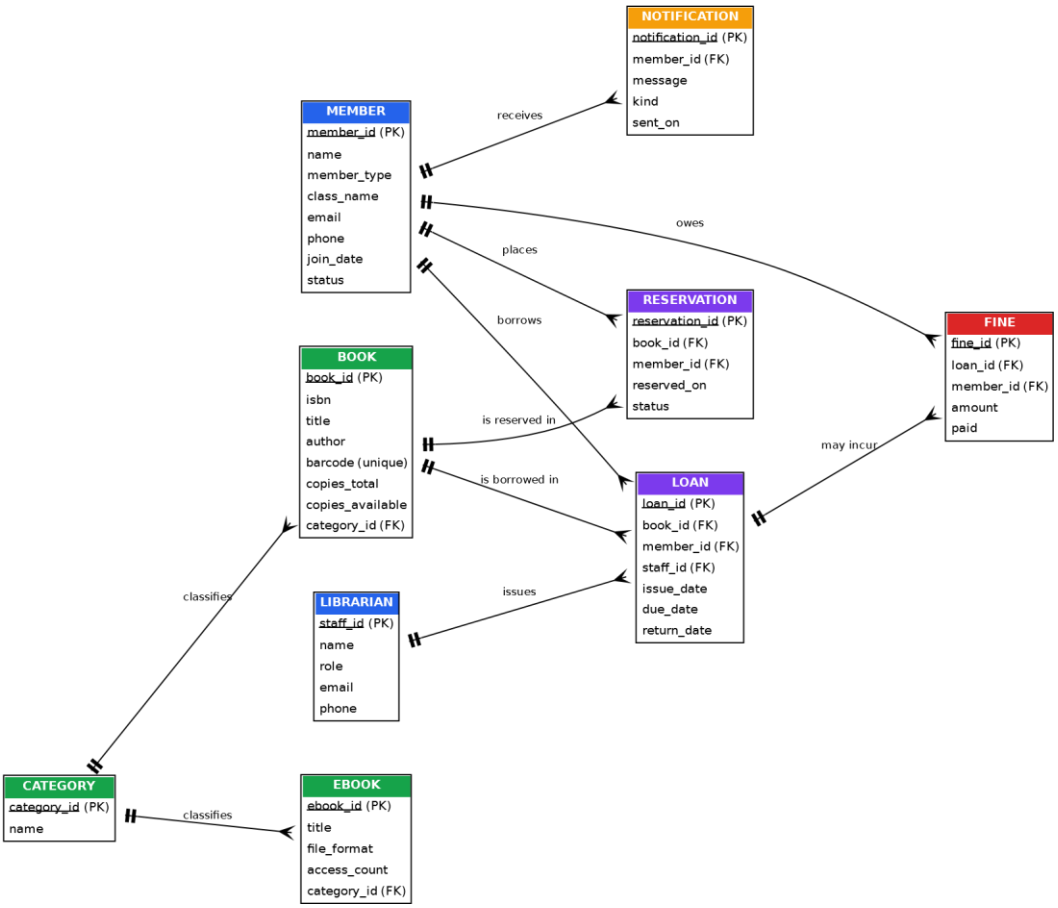



**Figure 6:** Entity Relationship Diagram.

## 3.5 Class Diagram

The class diagram contains eleven classes. Member and Librarian inherit from the abstract Person. Library is a service (facade) class that applies the business rules and talks to the database; the interface only calls this class.

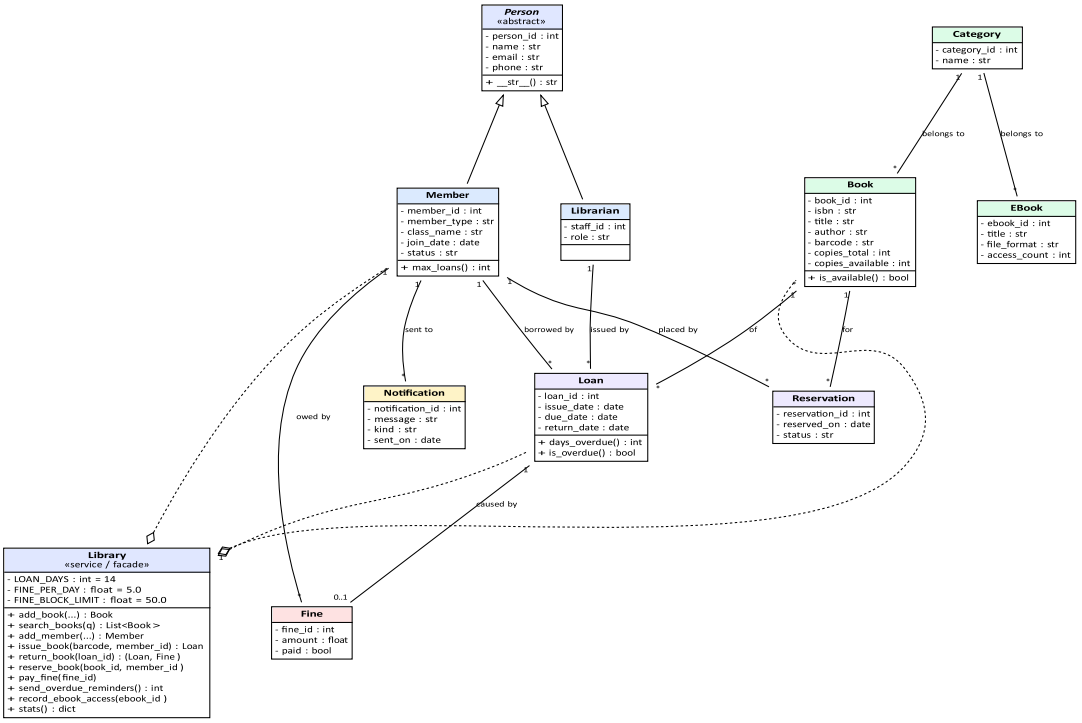



**Figure 7:** Class Diagram.

## 3.6 Object Diagram

The object diagram is a snapshot of the system on 25 October 2026. Aarav holds an overdue copy of *The Alchemist*; Diya returned the same title late and owes a BDT 25 fine; and the only copy of *The Selfish Gene* is on loan while Diya waits in the reservation queue.

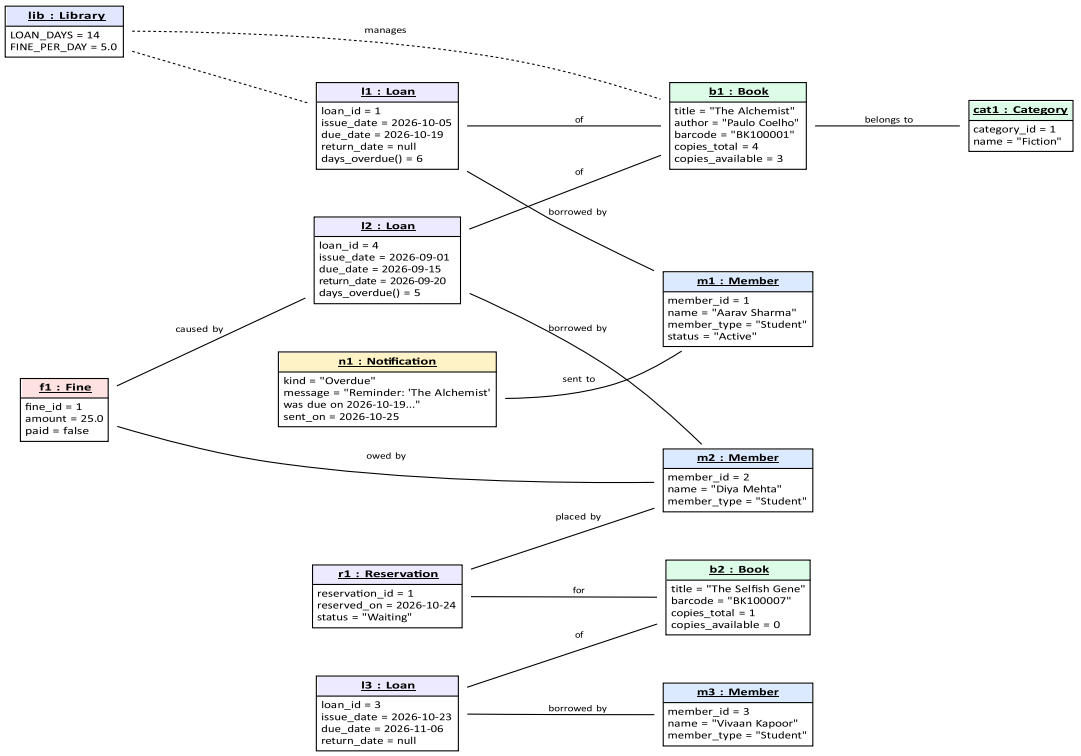


**Figure 8:** Object Diagram: snapshot of the library on 25 October 2026.

### 3.7 Architecture

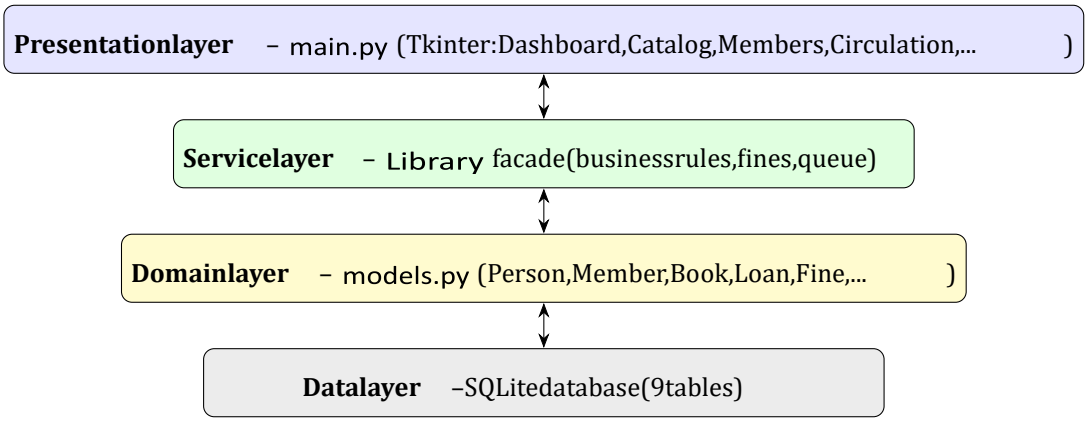



**Figure9:** Four-layerarchitectureofSmartShelf.

## 4 Prototype Implementation

### 4.1 Technology and Structure

The prototype is written in Python 3 with Tkinter (graphical interface) and SQLite (database); only the standard library is used. Files: models.py (domain classes), library.py (service layer and persistence), main.py (interface), test_library.py (rule tests). To run: python3 main.py (demo data are created on first start).

### 4.2 Classes Used by the Interface

The assignment requires at least five classes from the class diagram in the interface. The prototype uses eight (plus the Library service):

| Class | Interface tab | How it is used |
|---|---|---|
| Book | Catalog | Rows of the search list; add-book form creates a Book |
| Category | Dashboard | Donut chart of copies per category |
| Member (extends Person) | Members, Circulation | Member list; loan limit from max_loans |
| Loan | Circulation, Dashboard | Active loans and overdue list; days overdue() |
| Reservation | Reservations | Queue with status Waiting / Ready |

| | | |
|---|---|---|
| Fine | Fines | Unpaid and paid fines; “Mark as paid” |
| EBook | E-Books and Alerts | Access counter |
| Notification | E-Books and Alerts | Simulated SMS log |
| Librarian | Header | Logged-in user (full login planned in Sprint 3) |

**Table 7:** Mapping from class diagram to the user interface.

## 4.3 Screenshots

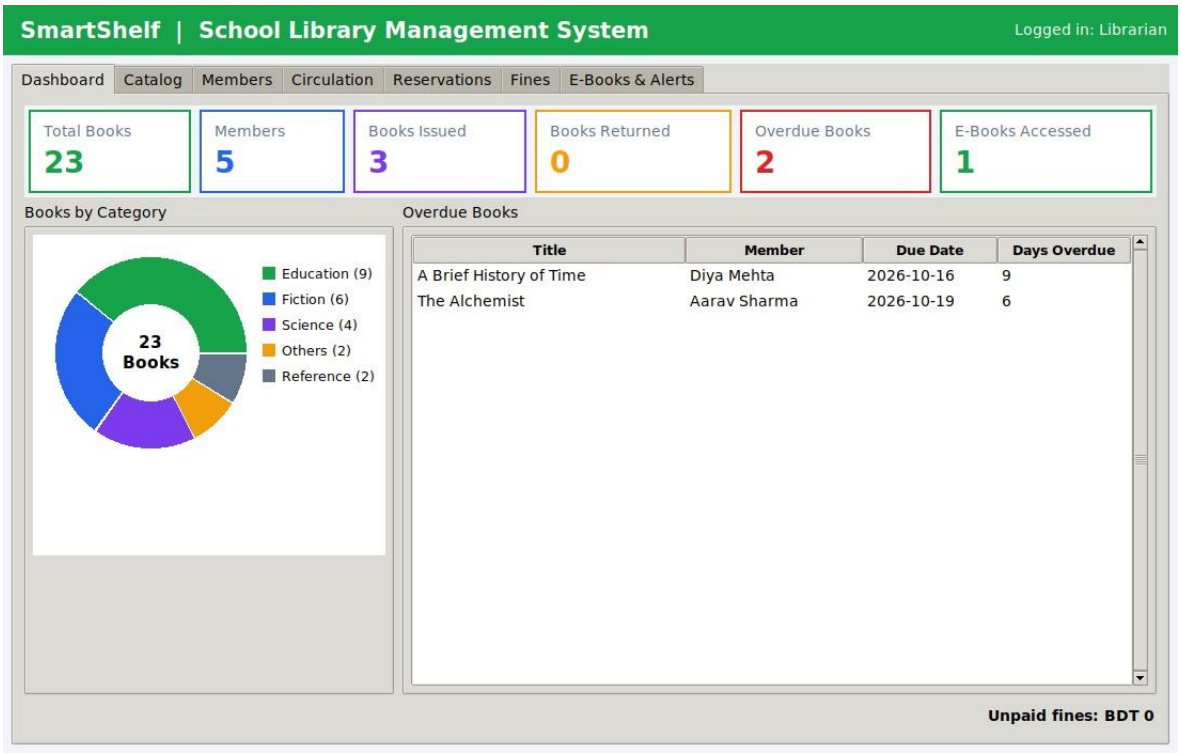



**Figure 10:** Dashboard: KPI cards, books by category, overdue books.

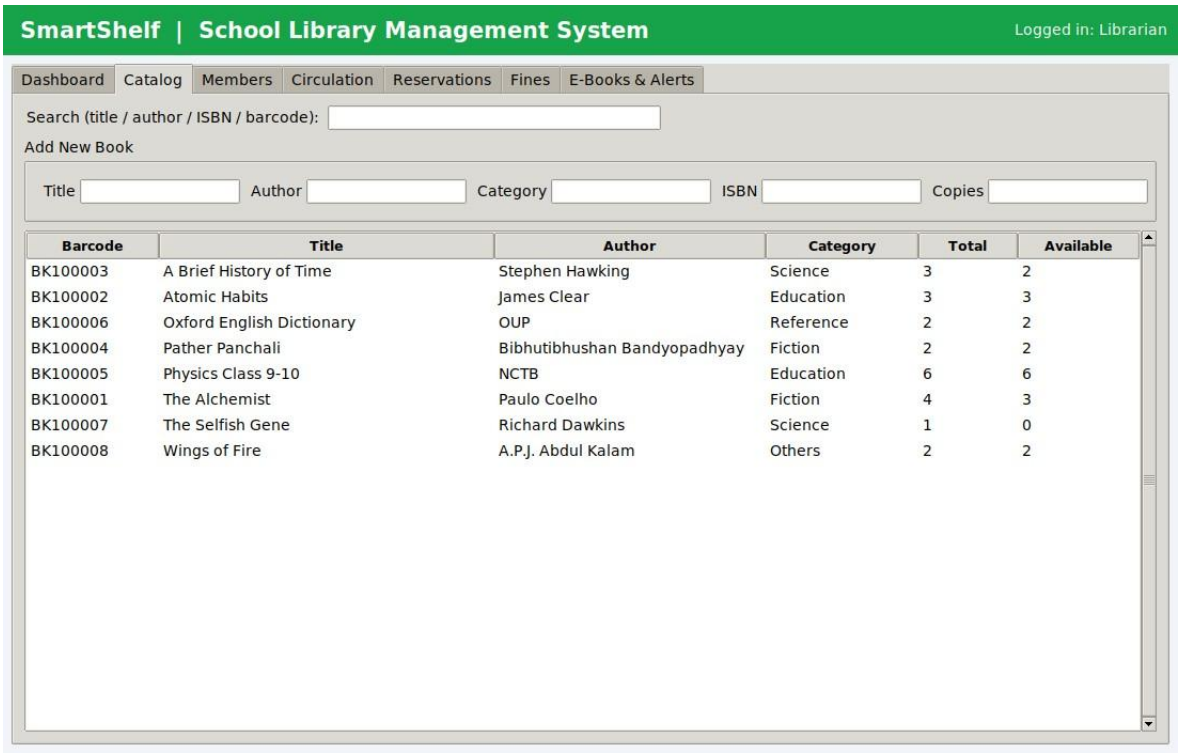



**Figure 11:** Catalog tab: search and add books.

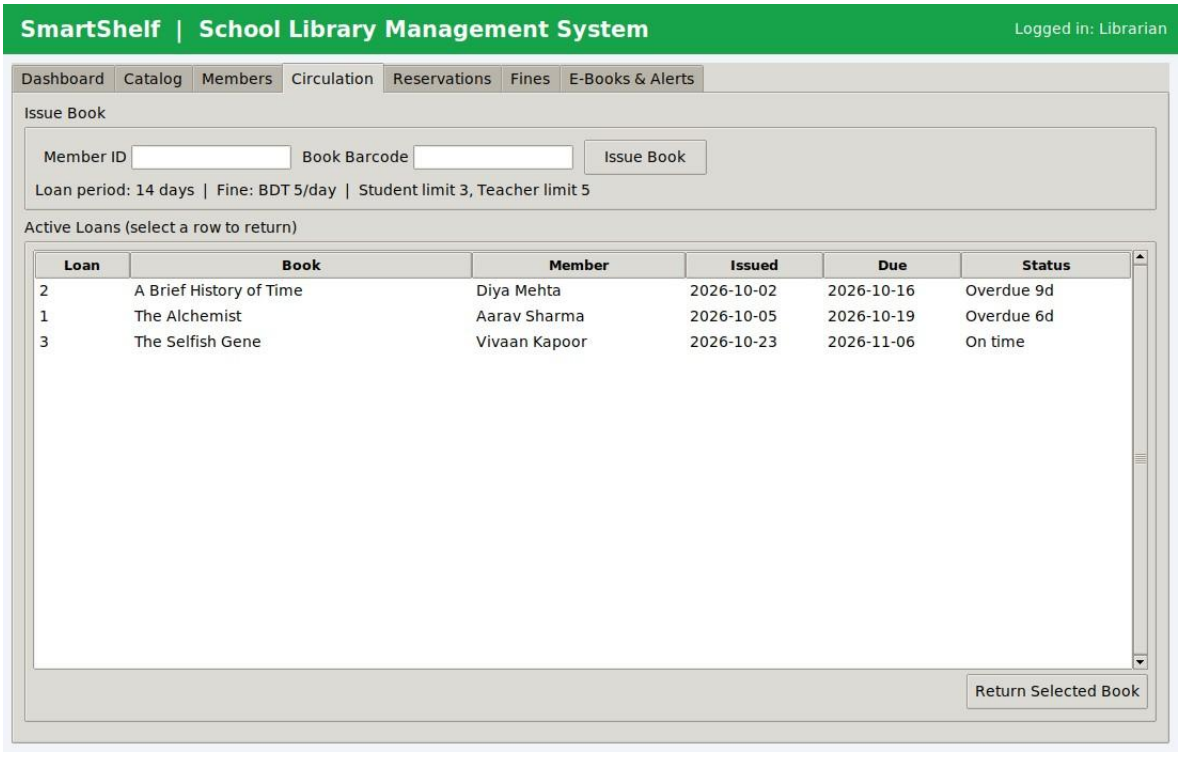



**Figure 12:** Circulation tab: issue book and return selected loan.

## 4.4 Selected Source Code

The Library.issue book method applies all issue rules from the activity diagram:

```
    def issue_book(self, barcode, member_id, staff_id=None, today=None):
        today = today or date.today()

        member = self.get_member(member_id) if not member:
            raise LibraryError("Member not found.")
        if member.status != "Active":
            raise LibraryError("Member account is not active.")
        book = self.get_book_by_barcode(barcode) if not book:
            raise LibraryError("Book barcode not found.")
        if self.unpaid_fines_total(member_id) >= self.FINE_BLOCK_LIMIT:
            raise LibraryError("Unpaid fines exceed the limit. Please pay fines first.
                ")
        if self.active_loans_count(member_id) >= member.max_loans:
            raise LibraryError(f"Loan limit reached ({member.max_loans} books).")
        if self._one("SELECT 1 FROM loan WHERE book_id=? AND member_id=? AND return_date IS NULL",
            book.book_id, member_id):
            raise LibraryError("This member already has this book.")
        ready_others = self._one(
            "SELECT COUNT(*) FROM reservation WHERE book_id=? AND status='Ready' AND member_id<>?",
            book.book_id, member_id)[0]
        if book.copies_available - ready_others <= 0:
            raise LibraryError("Held for another member's reservation." if ready_others else "No copy available. You can
                reserve this book.")
        due = today + timedelta(days=self.LOAN_DAYS) cur = self.db.execute(
            "INSERT INTO loan(book_id,member_id,staff_id,issue_date,due_date) VALUES (?,?,?,?,?)",
            (book.book_id, member_id, staff_id, today.isoformat(), due.isoformat())) self.db.execute("UPDATE book SET
        copies_available=copies_available-1 WHERE book_id=?",
                                (book.book_id,))
        self.db.execute("UPDATE reservation SET status='Fulfilled' WHERE book_id=? AND member_id=?"
                                " AND status='Ready'", (book.book_id, member_id))
        self.db.commit() return self.get_loan(cur.lastrowid)
```

```
    def return_book(self, loan_id, today=None): """Returns (loan,
        fine_or_None)."""" today = today or date.today() loan =
        self.get_loan(loan_id) if not loan or loan.return_date:
            raise LibraryError("Active loan not found.")
        self.db.execute("UPDATE loan SET return_date=? WHERE loan_id=?", (today. isoformat(), loan_id))
        self.db.execute("UPDATE book SET copies_available=copies_available+1 WHERE book_id=?",
                        (loan.book_id,))
        loan.return_date = today.isoformat() fine = None days =
        loan.days_overdue() if days > 0:
            amount = days * self.FINE_PER_DAY
            cur = self.db.execute("INSERT INTO fine(loan_id,member_id,amount) VALUES
                (?,?,?)",
                                                  (loan_id, loan.member_id, amount))
            fine = Fine(cur.lastrowid, loan_id, loan.member_id, amount, False, loan.member_name, loan.book_title)
        self._promote_reservation(loan.book_id, today) self.db.commit() return
        loan, fine
```

The Library.return _book method closes the loan, creates the fine and promotes the next reservation:

# Appendix A: How to Run

1. Install Python 3.10 or later (Tkinter is included on Windows and macOS; on Ubuntu run sudo apt install python3-tk).
2. Open a terminal in the src folder and run python3 main.py.
3. Run the rule tests with python3 test _library.py.

# Appendix B: Domain Classes (models.py)

```
"""Domain classes for SmartShelf (these match the UML Class Diagram)."""
from dataclasses import dataclass
from datetime import date


class Person:
    """Abstract base class for people in the system."""
    def __init__(self, person_id, name,
    email="", phone=""):
        self.person_id = person_id self.name =
        name self.email = email self.phone =
        phone

    def __str__(self): return f"{self.name} <{self.email}>"


class Member(Person):
    MAX_LOANS = {"Student": 3, "Teacher": 5}

    def __init__(self, member_id, name, member_type="Student", class_name="", email="", phone="",
                 join_date=None, status="Active"):
        super().__init__(member_id, name, email, phone) self.member_id = member_id
        self.member_type = member_type self.class_name = class_name self.join_date =
        join_date or date.today().isoformat() self.status = status

    @property def max_loans(self): return
    self.MAX_LOANS.get(self.member_type, 3)


class Librarian(Person):
    def __init__(self, staff_id, name, role="Librarian", email="", phone=""):
        super().__init__(staff_id, name, email, phone) self.staff_id = staff_id
            self.role = role                                    # "Librarian" or "Admin"


@dataclass    class    Category:
category_id: int name: str


@dataclass class Book: book_id: int isbn: str
title: str author: str category_id: int barcode:
str copies_total: int = 1 copies_available: int
= 1 category_name: str = ""
```

```
    def    is_available(self):
```